

这份讲义的核心观点是：现代机器学习的性能提升，不能只靠更好的算法，也不能只靠更快的硬件，而要在“统计效率”和“硬件效率”之间做权衡；而统计学习算法对精确性的容忍，让我们可以通过放松一致性、异步更新、模型复制、低精度计算等方式，大幅提升训练速度。

1. 核心矛盾：统计效率 vs 硬件效率

讲义一开始提出一个关键平衡：

Statistical Efficiency：统计效率

指的是算法需要多少步才能收敛。比如 SGD 训练模型，要迭代多少次 loss 才能降下来。

Hardware Efficiency：硬件效率

指的是每一步能不能在硬件上跑得快，比如能不能充分利用多核 CPU、NUMA 内存结构、SIMD 向量计算等。

通俗来说：

统计效率关注“走多少步到终点”；
硬件效率关注“每一步走得快不快”。

传统算法设计往往更重视数学上的收敛速度，但在现代硬件上，如果每一步都要等待、加锁、通信，反而会很慢。所以机器学习系统真正要优化的是：**总耗时 = 步数 × 每步耗时**。

2. 为什么并行计算变得这么重要？

讲义指出，现代硬件有三个重要趋势：

1. 单核 CPU 不再显著变快
2. 芯片里有越来越多的小核心
3. 内存访问变得不均匀，NUMA 越来越重要
4. SIMD/SIMT 让一个指令可以同时处理多个数据

也就是说，过去我们可以等 CPU 自动变快，现在不行了。性能提升越来越依赖于：**你能不能把任务拆开，让很多核心同时干活。**

通俗理解：

以前是一个工人越来越强；
现在是工人数量越来越多。

但问题是：你得重新设计工作流程，否则人多反而互相等、互相堵。

3. 机器学习为什么天然适合并行？因为 SGD 是“海量小任务”

讲义用 SGD 作为例子。很多机器学习问题可以写成：

最小化所有样本上的损失函数之和。

SGD 的做法是：每次抽一个样本或一小批样本，估计梯度，然后更新模型参数。

通俗来说：

模型训练不是一次性算完，而是做几十亿次“小修小补”：

看一个样本，算一下方向，调整一点参数，再看下一个样本。

这带来一个天然机会：这些“小任务”看起来可以并行。但问题是，大家都要更新同一个模型参数，所以会出现冲突。

4. 多核并行的经典问题：加锁会拖慢系统

如果多个核心同时更新同一个参数，传统计算机科学会说：要加锁。

也就是每个核心更新前先问一句：

“现在轮到我了么？”

这样可以保证更新顺序正确，但代价很高。讲义里提到，锁协议本身可能需要很多 CPU 周期；当核心数变多，通信和等待的成本会快速上升。

通俗理解：

如果 100 个人都要在同一个白板上写东西，传统方式是每个人排队拿笔。

结果大家都在等，白板虽然没乱，但效率极低。

所以，传统加锁方式下，SGD 甚至可能出现一个反直觉现象：

核心越多，训练越慢。

5. Hogwild!：大胆忽略锁，反而更快

讲义中最重要的案例是 **Hogwild!**。

Hogwild! 的思想非常“反传统”：

不加锁，大家直接更新模型参数。

这听起来很危险，因为参数可能被多个线程同时修改，产生冲突。但讲义指出，对于很多稀疏的机器学习问题，即使不加锁，SGD 仍然可以收敛到正确答案，而且收敛速度差不多。

通俗理解：

如果 100 个人都在白板上写字，确实偶尔会互相覆盖。

但如果每个人写的位置大多不同，而且整体方向是对的，那最后白板上的答案仍然能逐渐变好。

与其为了避免一点点冲突让所有人排队，不如允许少量混乱，换取巨大速度提升。

这就是讲义反复强调的思想：

统计学习算法对“精确顺序”的要求没有传统数据库那么高。

数据库里一笔钱不能多扣或少扣；但机器学习里，一次参数更新稍微乱一点，可能没关系，因为训练本来就是带噪声的。

6. 更大的趋势：放松一致性，换取性能

Hogwild! 不是孤立案例。讲义把它上升为一个更大的系统设计原则：

Relaxing consistency to be architecturally aware can be a big performance win.

也就是：

适当放松一致性，并根据硬件结构设计系统，可以获得巨大的性能提升。

通俗解释：

不要死守“每一步都完全精确、完全同步”。

对机器学习来说，有时候“差不多但快很多”比“完全精确但慢很多”更有价值。

这对理解现代深度学习训练很重要。很多分布式训练、异步训练、参数服务器、低精度训练，本质上都在做类似权衡。

7. NUMA：不是所有内存访问都一样快

第二个硬件趋势是 **NUMA, Non-Uniform Memory Access**，即非统一内存访问。

简单说，现代服务器里可能有多个 CPU socket，每个 socket 附近有自己的内存。一个核心访问“离自己近”的内存会快，访问“远处”的内存会慢。

通俗理解：

就像办公室里每个小组旁边都有自己的资料柜。

拿自己旁边资料柜里的文件很快；

去别的小组那边拿文件就慢，还可能堵路。

这对机器学习系统很关键。因为如果所有核心都频繁访问同一个模型参数，就会造成跨 socket 的通信和缓存一致性开销，导致性能下降。

8. 模型复制：少通信，提升硬件效率

为了解决 NUMA 和跨核心通信问题，讲义介绍了 **Model Replication**，模型复制。

几种策略包括：

1. **Per Machine**：整台机器共享一个模型
2. **Per Core**：每个核心一份模型
3. **Per Node**：每个 NUMA 节点一份模型

共享一个模型的好处是统计效率高，因为所有更新都立刻汇聚到同一个模型上；缺点是硬件效率低，因为通信和缓存冲突严重。

每个核心一份模型的好处是硬件效率高，因为几乎不用抢同一个模型；缺点是统计效率低，因为各自训练出来的模型需要合并，信息同步慢。

所以真正的问题是：

模型到底复制到什么粒度，才能在统计效率和硬件效率之间取得平衡？

讲义里的 DimmWitted 系统就是在探索这种权衡，包括访问方式、模型复制方式、数据复制方式等，并指出这种设计可能比传统选择快很多。

9. SIMD：一个指令同时处理多个数据

第三个趋势是 **SIMD, Single Instruction Multiple Data**。

普通计算是：

一次加两个数。

SIMD 是：

一条指令同时加多组数。

比如普通加法一次算：

$$4 + 4 = 8$$

SIMD 可以一次算：

$$1+2, 2+4, 3+6, 4+8$$

通俗理解：

普通计算像一个厨师一次切一根胡萝卜；
SIMD 像一个切片机，一刀下去切一排胡萝卜。

这对机器学习很重要，因为矩阵乘法、向量加法、梯度计算中有大量重复操作，非常适合 SIMD。

10. 精度 vs 并行度：低精度可以换来更高吞吐

讲义进一步指出，SIMD 中存在一个重要权衡：

数字精度越低，同一个寄存器里能塞进的数据越多，并行度越高。

例如：

1. 32-bit float：精度高，但一次能并行处理的数据少
2. 16-bit int/float：精度较低，但并行度更高
3. 8-bit int：精度更低，但吞吐更高

通俗理解：

一个箱子容量固定。
如果每个数字都很“大”，箱子里装得少；
如果每个数字更“小”，箱子里能装更多，机器一次能处理更多数据。

这就是为什么现代深度学习会大量使用 FP16、BF16、INT8，甚至更低精度。

关键是：机器学习不一定每个数都需要 32 位高精度。

11. DMGC 模型：不同类型的数，可以用不同精度

讲义把训练中的数字分成四类：

1. **Dataset numbers**：数据集里的数
2. **Model numbers**：模型参数
3. **Gradient numbers**：梯度计算中的中间数
4. **Communication numbers**：不同 worker 之间通信的数

它提出一个 DMGC 模型，用来描述不同类别数字分别用什么精度。

例如：

```
D8 M16 G32f C16
```

意思是：

1. 数据用 8-bit
2. 模型用 16-bit
3. 梯度用 32-bit float
4. 通信用 16-bit

这个思想很重要：**低精度不是“一刀切”地把所有东西都降精度，而是要分类型设计。**

通俗理解：

做饭时，不是所有东西都要用同一种刀。

切肉、切菜、削皮、剁骨头，可以用不同工具。

训练模型也是一样，不同数字承担的功能不同，对精度的敏感度也不同。

12. 最后一层提醒：优化不是深度学习的完整抽象

讲义最后提出一个很有意思的观点：

Optimization is a leaky abstraction for deep learning.

意思是：我们不能只从“loss 下降得快不快”来理解深度学习。

有些方法可能让训练 loss 下降更慢，看起来优化更差，但泛化能力更好，也就是测试集表现更好。

通俗理解：

考试训练里，刷题速度最快的人，不一定真正理解得最好。

模型训练也是一样，训练误差降得最快，不一定代表最终泛化最好。

这对理解深度学习非常关键：训练不是单纯把 loss 压到最低，而是在优化速度、硬件效率、泛化能力之间做复杂权衡。

这份讲义真正想让你记住什么？

我觉得最重要的是下面五点：

第一，现代 AI 性能提升的核心是并行。

单核变快的红利变少后，必须靠多核、NUMA、SIMD、GPU、分布式系统来提升吞吐。

第二，机器学习允许“不完全精确”。

因为 SGD 本来就是随机近似算法，所以少量异步、冲突、低精度，不一定破坏结果。

第三，放松一致性可以换来巨大速度。

Hogwild! 的核心启发是：不要为了完美同步牺牲全部并行能力。

第四，算法设计必须理解硬件。

不是数学公式写完就结束了。模型参数放在哪里、是否复制、怎么通信、用什么精度，都会影响训练速度。

第五，优化 loss 不是全部。

深度学习中，训练速度、最终泛化、硬件效率之间存在复杂关系。某些“优化上看起来不完美”的方法，可能反而更实用。

总结

现代大模型训练并不是单纯的算法问题，而是算法、系统和硬件协同优化的问题。比如 SGD 本身具有随机近似特征，所以在并行训练中可以适当放松一致性，通过 Hogwild 式异步更新、模型复制、NUMA-aware 设计和低精度计算，在统计效率和硬件效率之间做权衡。这也是为什么现代大模型训练会大量依赖分布式并行、混合精度、通信压缩和硬件感知的系统设计。